Lab Report-Individual

Coding Challenge: Exploring .NET MAUI for Cross-Platform App Development

In this coding challenge, you'll explore **.NET MAUI (Multi-platform App UI)**, a powerful framework for building native, cross-platform applications for iOS, Android, macOS, and Windows with a single codebase. Through these tasks, you will gain practical experience in creating dynamic user interfaces, handling events, and customizing app behavior using .NET MAUI features. By completing this challenge, you'll strengthen your skills in cross-platform app development and gain insights into how .NET MAUI can be used for efficient application development

(HINT: YOU CAN MODIFY THE CONTENT BY REFERRING TO .NET MAUI APPS)

Tasks

1. **Display a Label**
   - Provide a screenshot of your .NET MAUI app running in the emulator or on a device, showing a label with text.
2. **Customize Label Appearance**
   - Modify the label's text color, font size, and alignment. Provide a screenshot of the updated label.
3. **Present Formatted Text**
   - Use formatted text in a label to display multiple font styles and colors. Provide a screenshot of the formatted label.
4. **Add a Button**
   - Provide a screenshot of your .NET MAUI app running in the emulator or on a device, showing a button.
5. **Handle Button Clicks**
   - Implement a click event for the button that displays a message. Provide a screenshot of the app after the button is clicked.
6. **Customize Button Appearance**
   - Change the button's background color, border, and text style. Provide a screenshot of the customized button.
7. **Use an Entry**
   - Provide a screenshot of an entry field in your .NET MAUI app.
8. **Respond to Text Changes in Entry**
   - Implement an event handler to respond when text in the entry field changes. Provide a screenshot of the app demonstrating this functionality.

9. **Customize Entry Behavior**
    - Modify the entry to include placeholder text, set input constraints (e.g.,numericonly), and change its appearance. Provide a screenshot of the customized entry.
10. **Add an Editor**
    - Add an editor component for multi-line text input. Provide a screenshot of the editor in your .NET MAUI app.

Table Of Contents

Contents

1. Display a Label 1
2. Customize Label Appearance 2
3. Present Formatted Text 3
4. Add a Button 3
5. Handle Button Clicks 4
6. Customize Button Appearance 7
7. Use an Entry 8
8. Respond to Text Changes in Entry 10
9. Customize Entry Behavior 12
10. Add an Editor 15

Tasks

1. **Display a Label**
   - Provide a screenshot of your .NET MAUI app running in the emulator or on a device, showing a label with text.

```
<!-- Label -->
<Label
    Text="Hello, World!"
    Style="{StaticResource Headline}"
    SemanticProperties.HeadingLevel="Level1" />

<Label
    Text="Welcome to &#10;.NET Multi-platform App UI"
    Style="{StaticResource SubHeadline}"
    SemanticProperties.HeadingLevel="Level2"
    SemanticProperties.Description="Welcome to dot net Multi platform App U I" />
```

Figure 1: Diagram above shows the code snippet for displaying the label

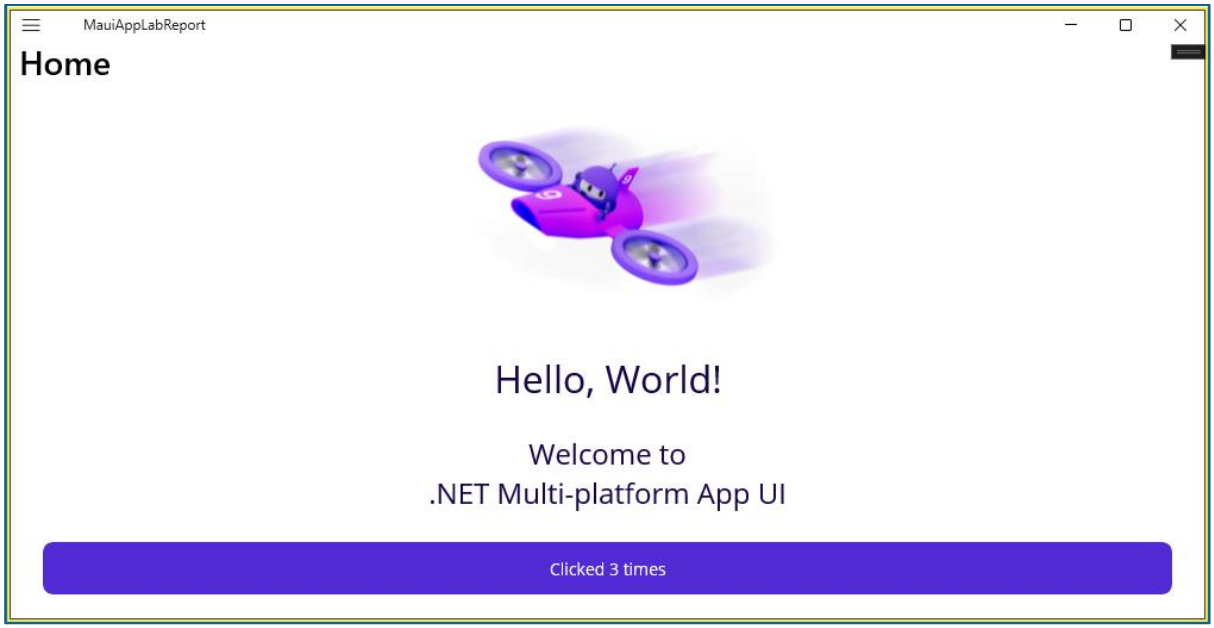

Figure 2: Diagram above shows the output for the above code by displaying the label

2. **Customize Label Appearance**
   - Modify the label's text color, font size, and alignment. Provide a screenshot of the updated label.

```
<Image
    Source="hello_world.png"
```

```
<Label
    Text="Hello, World! 😊"
    FontSize="24"
    TextColor="DarkCyan"
    HorizontalOptions="Center"
    VerticalOptions="Center"
/>
```

```
<!-- Welcome Label -->
<Label
    Text="Welcome to &#10;.NET Multi-platform App UI"
    TextColor="Purple"
    FontSize="24"
    FontFamily="Calibri"
    HorizontalOptions="Center"
    VerticalOptions="Center"
    Style="{StaticResource SubHeadline}"
    SemanticProperties.HeadingLevel="Level2"
    SemanticProperties.Description="Welcome to dot net Multi platform App U I" />
```

Figure 3: Diagram above shows the code snippet for customizing the label appearance and change of welcome image.

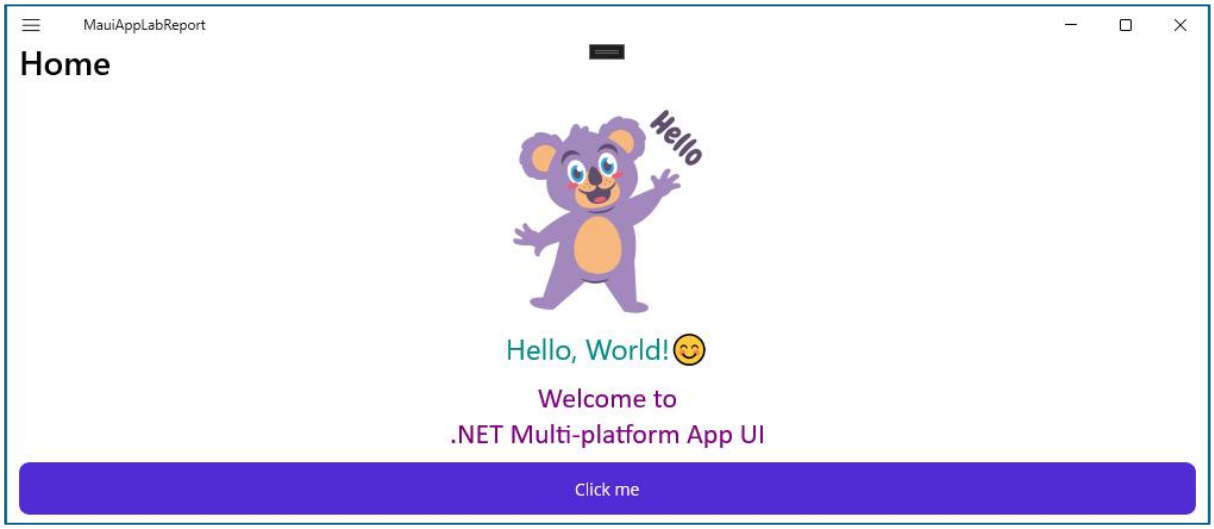

Figure 4: Diagram above shows the output for the above code by displaying the customized label and change of welcome image.

3. **Present Formatted Text**
   - Use formatted text in a label to display multiple font styles and colors. Provide a screenshot of the formatted label.

```
<!-- Formatted Label -->
<Label
    HorizontalOptions="Center"
    VerticalOptions="Center">
   <Label.FormattedText>
      <FormattedString>
         <Span Text="Hello, " FontSize="24" TextColor="DarkCyan" FontAttributes="Bold" />
         <Span Text="World!" FontSize="24" TextColor="Red" FontAttributes="Italic" TextDecorations="Underline" />
         <Span Text="😊" FontSize="24"/>
      </FormattedString>
   </Label.FormattedText>
</Label>
```

```
<!-- Welcome Label -->
<Label
    Text="Welcome to &#10;.NET Multi-platform App UI"
    TextColor="Purple"
    FontSize="24"
    FontFamily="Calibri"
    FontAttributes="Italic"
    HorizontalOptions="Center"
    VerticalOptions="Center"
    Style="{StaticResource SubHeadline}"
    SemanticProperties.HeadingLevel="Level2"
    SemanticProperties.Description="Welcome to dot net Multi platform App U I" />
```

Figure 5: Diagram above shows the code snippet or formatted text in the label

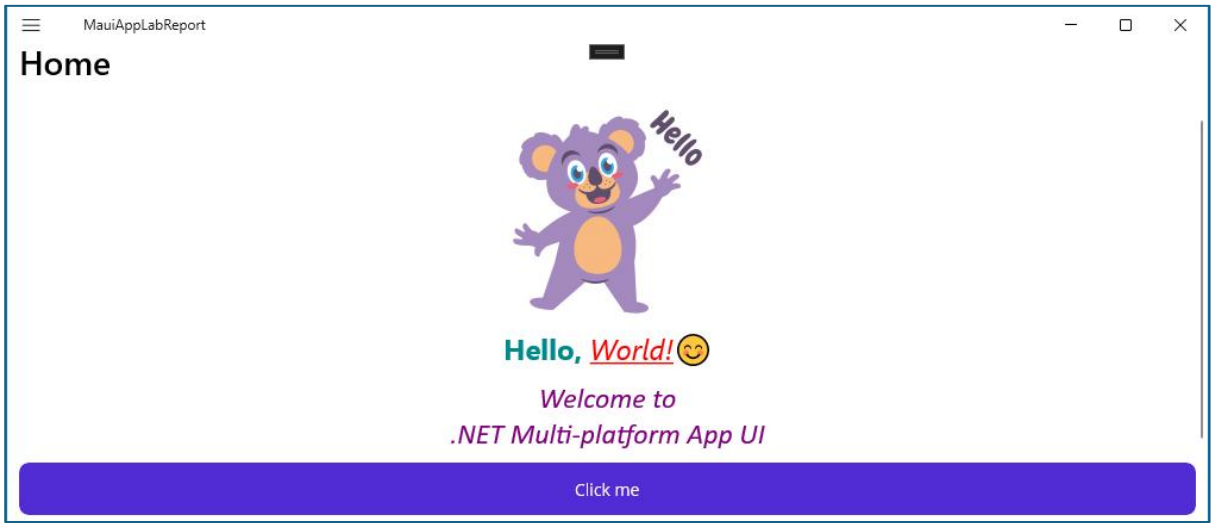

Figure 6: Diagram above shows the output for the above code by displaying the formatted text in the label

4. **Add a Button**

- Provide a screenshot of your .NET MAUI app running in the emulator or on a device, showing a button.

```
<Button
    Text="Click to Rotate Text!"
    VerticalOptions="Center"
    HorizontalOptions="Fill"
    Clicked="OnRotateButtonClicked" />
```

Figure 7 : Diagram above shows the code snippet for adding a button

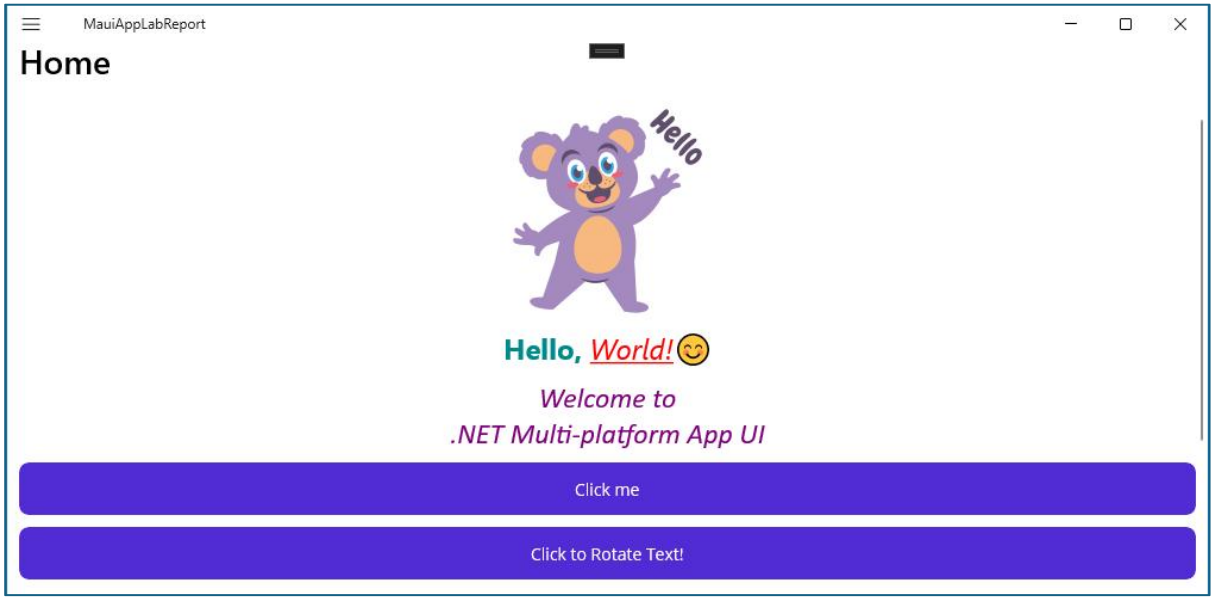

Figure 8: Diagram above shows the output for the above code on the added button

5. Handle Button Clicks

- Implement a click event for the button that displays a message. Provide a screenshot of the app after the button is clicked.

```
namespace MauiAppLabReport
{
    14 references
    public partial class MainPage : ContentPage
    {
        int count = 0;
        private bool isRotated = false; // Variable to track rotation state

        0 references
        public MainPage()
        {
            InitializeComponent();
        }
```

```
<!-- Label to Rotate -->
<Label
x:Name="RotatingLabel"
Text="Click the Button above"
FontSize="18"
VerticalOptions="Center"
HorizontalOptions="Center" />
```

```
private void OnRotateButtonClicked(object sender, EventArgs e)
{
    // Check current rotation state and toggle
    if (isRotated)
    {
        RotatingLabel.Rotation = 0; // Reset rotation
    }
    else
    {
        RotatingLabel.Rotation = 180; // Rotate by 180 degrees
    }

    isRotated = !isRotated; // Toggle rotation state
}
```

Figure 9: Diagram above shows the code snippet for handling button that will rotate the text (label) for 180° when the button is clicked

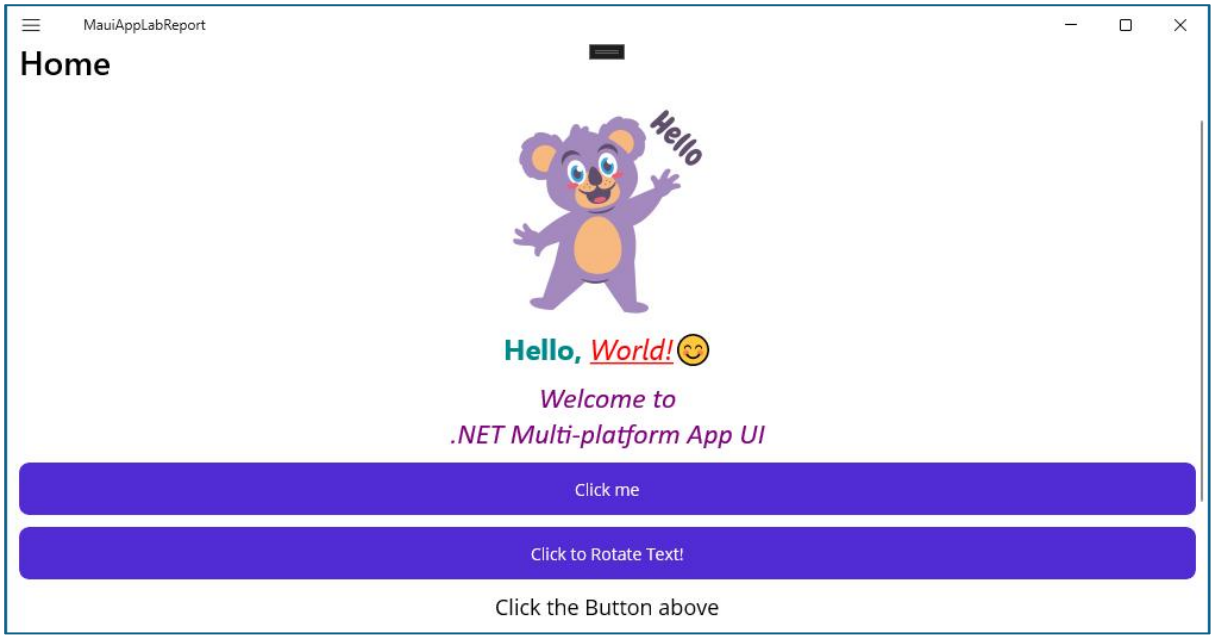

Figure 10: Diagram above shows the output handling the button clicks to rotate with the label that corresponds to the button.

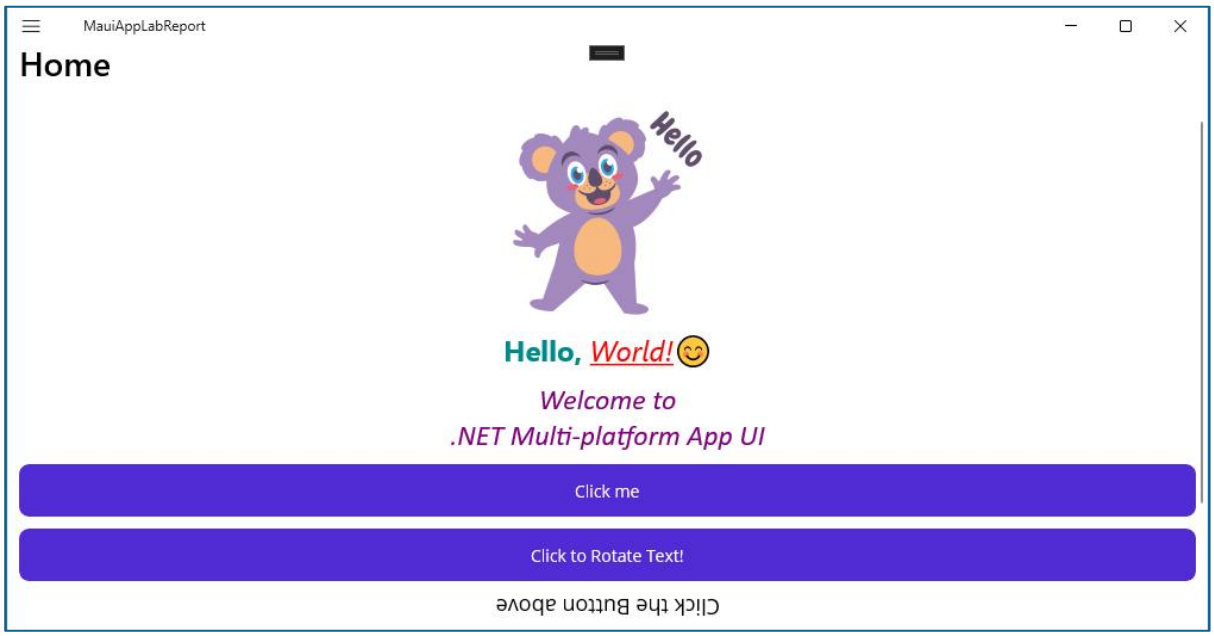

Figure 11: Diagram above shows the output of the rotated text (label) after "Clicked to Rotate Text" button is clicked

6. **Customize Button Appearance**
   - Change the button's background color, border, and text style. Provide a screenshot of the customized button.

```
<!-- Rotate Button with Border -->
<Button
    Text="Click to Rotate Text!"
    VerticalOptions="Center"
    FontAttributes="Italic,Bold"
    BackgroundColor="DarkCyan"
    BorderColor="DarkBlue"
    BorderWidth="4"
    HorizontalOptions="Fill"
    Clicked="OnRotateButtonClicked" />
```

```
<!-- Label to Rotate -->
<Label
x:Name="RotatingLabel"
Text="Click the Button above"
FontSize="18"
TextColor="SaddleBrown"
FontAttributes="Italic,Bold"
TextDecorations="Underline"
VerticalOptions="Center"
HorizontalOptions="Center" />
```

Figure 12: Diagram above shows the code snippet of the both customized rotating button and label with border.

Figure 13: Diagram above shows the output of the both customized rotating button and label with border.

7. **Use an Entry**
   - Provide a screenshot of an entry field in your .NET MAUI app.

```
<!-- Entry -->
<Entry
Placeholder="Type here..."
HorizontalOptions="Center"
BackgroundColor="DarkBlue"
TextColor="Aqua"
FontSize="20"
VerticalOptions="Center"
WidthRequest="250" />
```

```
// Entry text changed event handler
0 references
private void OnEntryTextChanged(object sender, TextChangedEventArgs e)
{
    string oldText = e.OldTextValue; // Get the old text
    string newText = e.NewTextValue; // Get the new text
    System.Diagnostics.Debug.WriteLine($"Text changed from '{oldText}' to '{newText}'");
}


// Entry completed event handler
0 references
private void OnEntryCompleted(object sender, EventArgs e)
{
    string completedText = ((Entry)sender).Text; // Get the completed text
    System.Diagnostics.Debug.WriteLine($"Entry completed with: {completedText}");
}
```

Figure 14: Diagram above shows the code snippet of the entry field

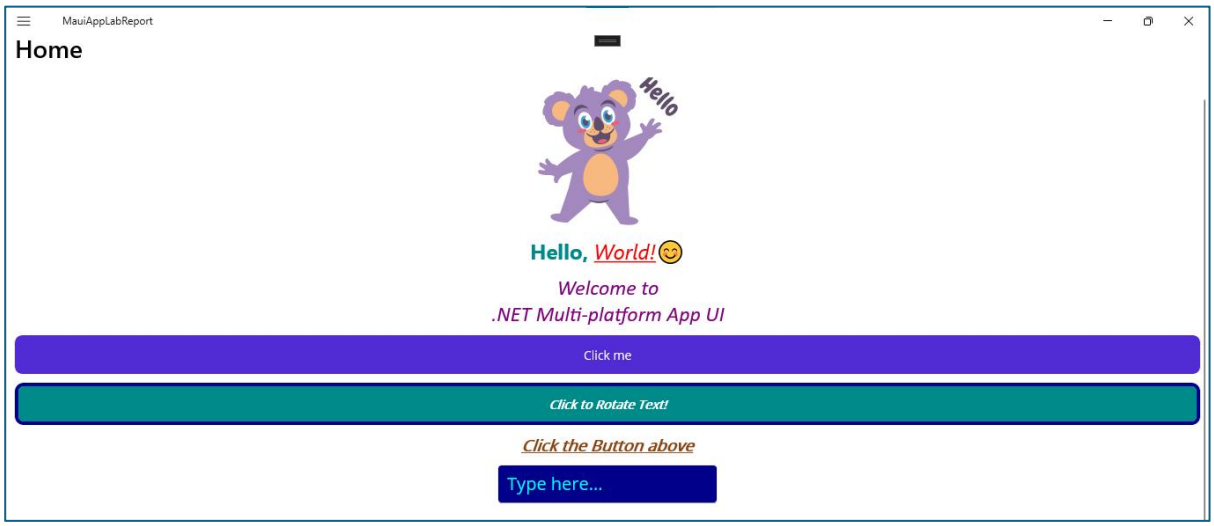

Figure 15: Diagram above shows the output of the entry of "Type here…"

Figure 16: Diagram above shows the output of the entry of "Type here…" changing to the entered text e.g. (Krisha)

8. **Respond to Text Changes in Entry**
   - Implement an event handler to respond when text in the entry field changes. Provide a screenshot of the app demonstrating this functionality.

```
<!--Entry -->
   <Entry Text="Type text will be displayed in uppercase when Enter is clicked."
   BackgroundColor="DarkBlue"
   TextColor="Aqua"
   FontSize="20"
   VerticalOptions="Center"
   WidthRequest="800"
   Completed="OnEntryCompleted" />
```

```
0 references
private void OnEntryCompleted(object sender, EventArgs e)
{
    var entry = (Entry)sender;
    entry.Text = entry.Text.ToUpper(); // Manually transform to uppercase on completion
}
```

Figure 17: Diagram above shows the code snippet of text changing to uppercase when enter is clicked.

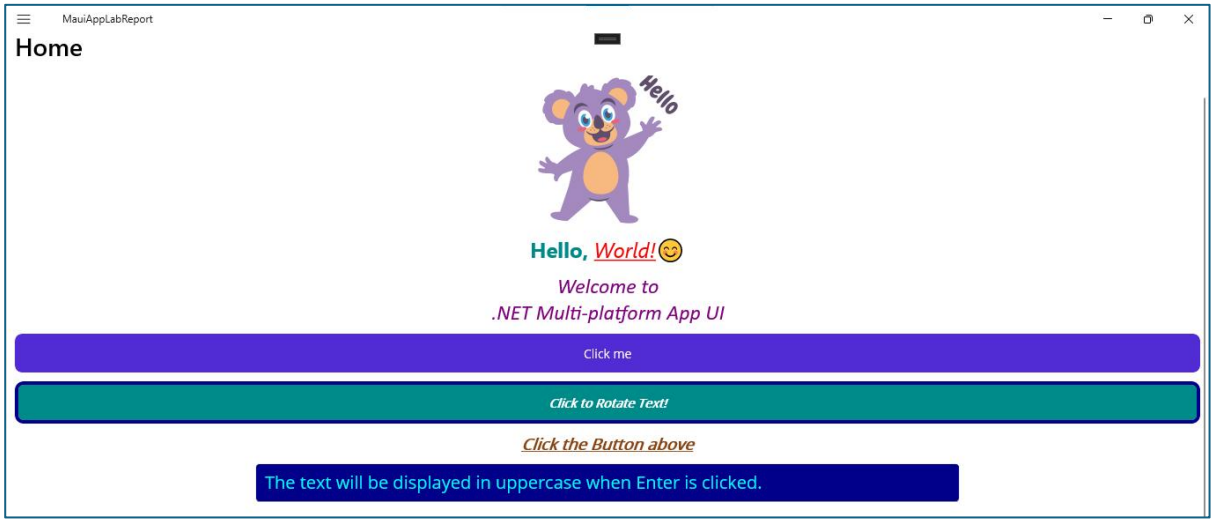

Figure 18: Diagram above shows the output with text entry that displays "This text will be displayed in uppercase when Enter is clicked".

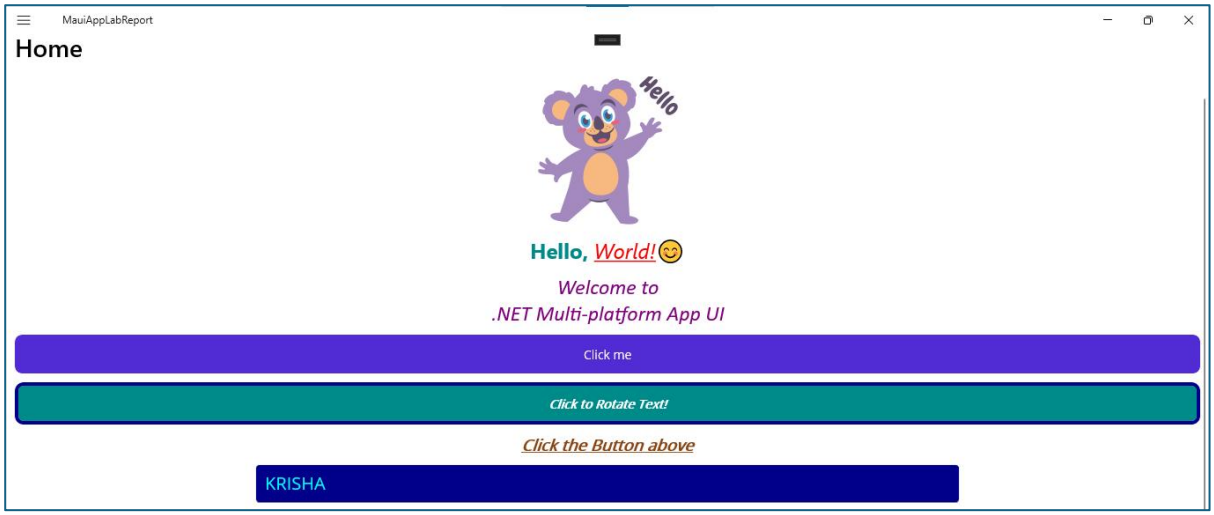

Figure 19: Diagram above shows the output with text entry that displays entered text in uppercase after clicking enter.

9. **Customize Entry Behavior**
   - Modify the entry to include placeholder text, set input constraints (e.g., numeric only), and change its appearance. Provide a screenshot of the customized entry.

```
<!-- Entry -->
<Entry
Placeholder="Enter numbers only..."
Keyboard="Numeric"
BackgroundColor="DarkBlue"
TextColor="Aqua"
FontSize="20"
FontAttributes="Bold"
Margin="20"
WidthRequest="300"
HorizontalOptions="Center"
VerticalOptions="Center"
Completed="OnEntryCompleted" />
```

```
0 references
private void OnEntryCompleted(object sender, EventArgs e)
{
    var entry = (Entry)sender;

    if (int.TryParse(entry.Text, out _)) // Ensure the input is numeric
    {
        DisplayAlert("Completed", $"You entered: {entry.Text}", "OK");
    }
    else
    {
        DisplayAlert("Error", "Please enter valid numbers only.", "OK");
        entry.Text = string.Empty; // Optionally, clear the input
    }
}
```

Figure 20: Diagram above shows code snippet for customized entry behavior which only allow numeric value and displays error message for non-numeric value.

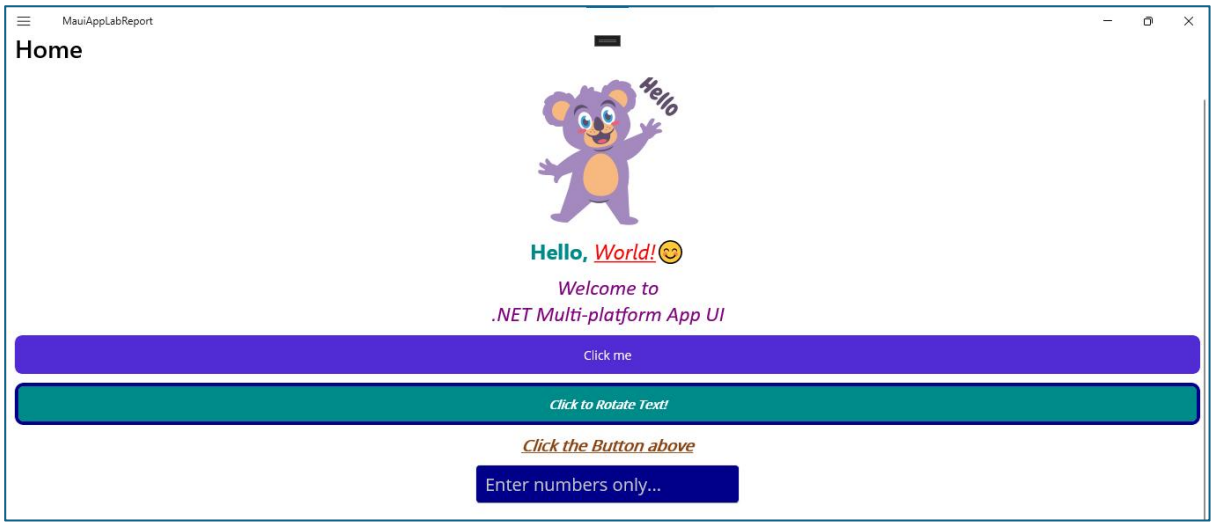

Figure 21: Diagram above shows output for customized entry behavior which only allow numeric value where the entry component displays " Enter number only…"

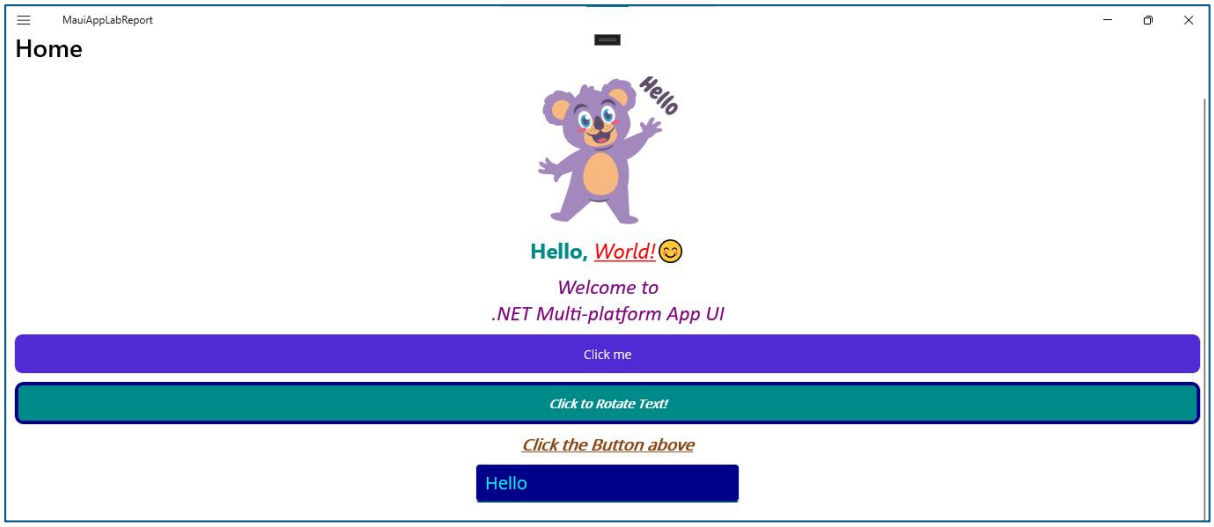

Figure 22: Diagram above shows output that a non numeric value of "Hello" is being entered.

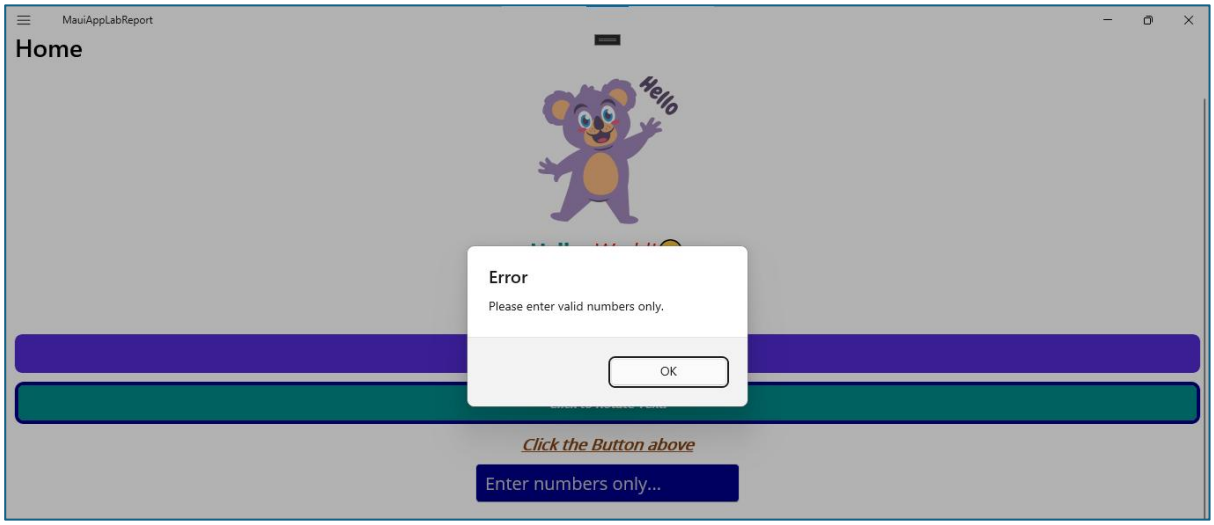

Figure 23: Diagram above shows output of error display since a non-numeric value of "Hello" was entered and the entry will be reset to "Enter numbers only...".

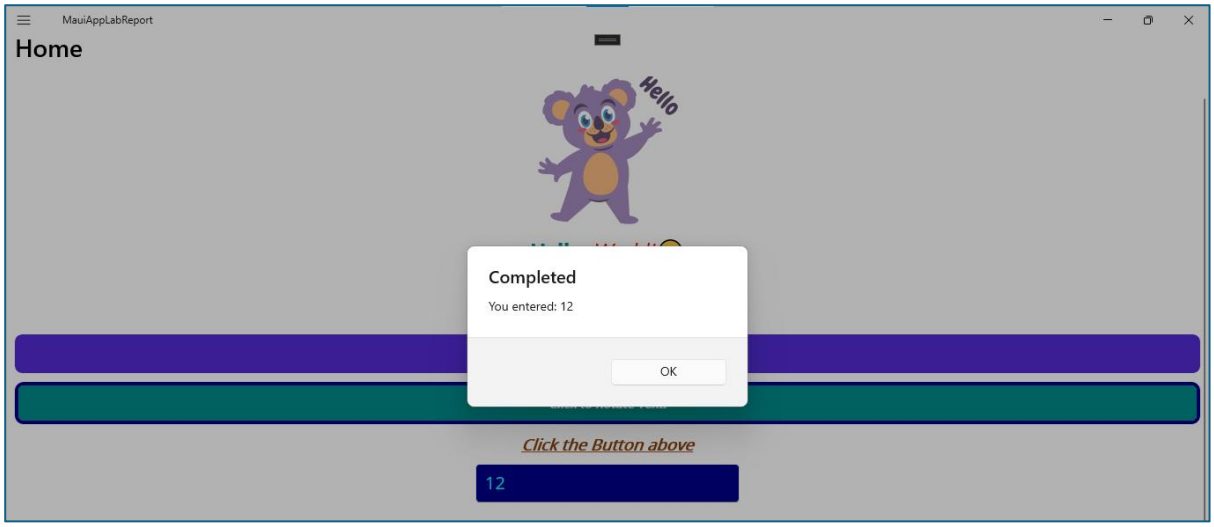

Figure 24: Diagram above shows output of completed display when numeric value of 12 was entered with the completed display which displays "You entered : 12".

10. Add an Editor

- Add an editor component for multi-line text input. Provide a screenshot of the editor in your .NET MAUI app.

```
<!-- Multi-line text input -->
<Editor
x:Name="MultiLineEditor"
Placeholder="Enter your multi-line text here..."
BackgroundColor="LightYellow"
TextColor="DarkBlue"
FontSize="16"
FontAttributes="Italic"
Margin="20"
HeightRequest="60"
WidthRequest="500"
HorizontalOptions="Center"
VerticalOptions="Center" />
```

```
<Button
Text="Done"
HorizontalOptions="Center"
VerticalOptions="Center"
Clicked="OnEditorDoneClicked" />
```

```
// Multi-text completion handler
0 references
private void OnEditorDoneClicked(object sender, EventArgs e)
{
    var text = MultiLineEditor.Text;

    // Display the entered text in an alert
    DisplayAlert("Editor Done", $"You entered: {text}", "OK");
}
```

Figure 25: Figure above shows the code snippet that demonstrates a multi-line text input using the Editor component, where the entered text is displayed in an alert upon clicking the 'Done' button."

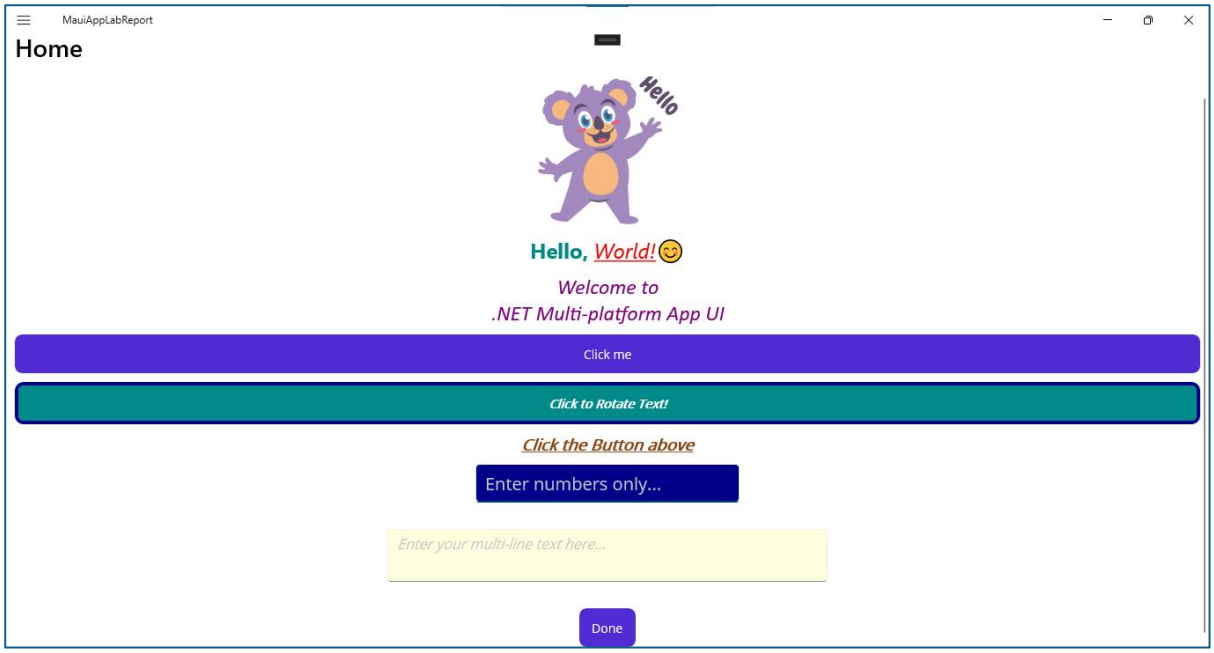

Figure 26: Figure above shows the output that with multi-line text input which display " Enter your multi-line text here…" and 'Done' button.

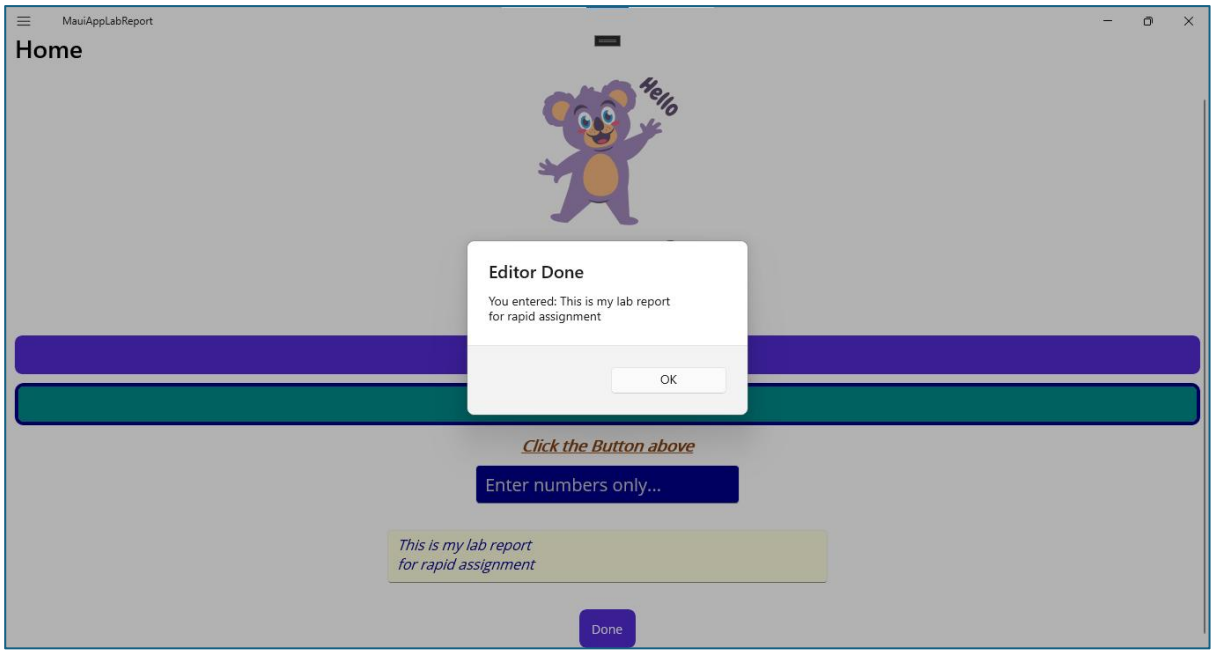

Figure 27: Figure above shows output that demonstrates a multi-line text input where the entered text is displayed in an alert upon clicking the 'Done' button."